

SDF Minimpl Project Report

CS24BTECH11036

Krishnan Ramanarayanan

April 2025

1 Aim of the Project

To modify the minimpl language to implement the division of 2 integers operation along with the exceptions - `DivisionByZero` and `GenericException`.

2 Formal Section

Overview

MiniML has been extended to include exception handling with custom exception types and control structures for throwing and catching exceptions. These features enable users to manage runtime errors more gracefully and define exceptions for specific use cases.

1. Division Operation

Division (`/`) is now supported as an arithmetic operation. If a division by zero occurs, the language raises the `DivisionByZero` exception.

<pre>10 / 2 ;; (* Output: 5 *) 10 / 0 ;; (* Output: Exception: Division by zero *)</pre>
--

2. New Exception Types

- **`DivisionByZero`**: Raised when division by zero occurs.
- **`GenericException of int`**: A generic exception carrying an integer value (positive or negative).

3. Throwing and Catching Exceptions

Raising Exceptions

```
raise DivisionByZero ;;
raise GenericException(1) ;;
raise GenericException(-2) ;;
```

Catching Exceptions

```
try raise DivisionByZero with DivisionByZero -> 5 ;;
try raise GenericException(42) with GenericException(42) ->
  ↪ 10 ;;
```

4. Example Interactions

```
MiniML> 3 + (if 5 < 6 then 10 else 100) ;;
- : int = 13

MiniML> let x = 14 ;;
x : int = 14

MiniML> let fact = fun f (n : int) : int is if n = 0 then 1
  ↪ else n * f (n-1) ;;
fact : int -> int = <fun>

MiniML> fact 10 ;;
- : int = 3628800

MiniML> raise DivisionByZero ;;
- : TVar = Exception: Division by zero

MiniML> raise GenericException(1) ;;
- : TVar = Exception: Generic exception: 1

MiniML> raise GenericException(-2) ;;
- : TVar = Exception: Generic exception: -2

MiniML> try raise DivisionByZero with DivisionByZero -> 5 ;;
- : int = 5

MiniML> try raise DivisionByZero with DivisionByZero -> -5
  ↪ ;;
- : int = -5

MiniML> try raise GenericException(42) with
  ↪ GenericException(42) -> 10 ;;
- : int = 10

MiniML> try raise GenericException(101) with
  ↪ GenericException(101) -> 20 ;;
- : int = 20
```

```

MiniML> try 10 / 0 with DivisionByZero -> -10 ;;
- : int = -10

MiniML> let fib = fun f (n : int) : int is
  if n < 21 then
    if n < 2 then n
    else f(n - 1) + f(n - 2)
  else raise GenericException(-32) ;;
fib : int -> int = <fun>

MiniML> try fib 30 with GenericException(-32) -> -32 ;;
- : int = -32

```

5. Nested Try-Catch Blocks

MiniML supports multiple levels of `try ... with` blocks:

```

MiniML> try
  try raise DivisionByZero with
    DivisionByZero -> 17
with
  DivisionByZero -> 108 ;;
- : int = 17

MiniML> try
  (try 10 / 0 with
    DivisionByZero -> raise (GenericException(1)))
with
  GenericException(1) -> -21 ;;
- : int = -21

```

6. Error Handling in Recursive Functions

Exception handling is integrated into recursive function calls:

```

MiniML> let fact = fun f (n : int) : int is
  if n < 0 then raise GenericException(-99)
  else if n = 0 then 1
  else n * f (n - 1) ;;

fact : int -> int = <fun>

MiniML> try fact (-3) with GenericException(-99) -> -99 ;;
- : int = -99

```

Conclusion

This documentation summarizes the new features added to MiniML, including:

- Custom exception types
- Division with automatic exception handling
- Throwing and catching exceptions
- Nested try-catch structures
- Robust error handling in recursive functions

3 READ ME

Introduction

This README section explains how to use the MiniML language with the new exception handling features, including custom exceptions like `DivisionByZero` and `GenericException`.

MiniML is a functional programming language that supports arithmetic, recursive functions, and exception handling. This version includes two new built-in exceptions:

- **DivisionByZero:** Raised when an attempt is made to divide by zero.
- **GenericException of int:** A generic exception that carries an integer argument.

With the new exception handling system, users can now raise and catch exceptions within their programs.

Installation

To get started with MiniML, clone the repository <https://github.com/Rmuk655/plzoo> and build the environment.

If you are under the plzoo directory, run:

```
python ./buildRun.py src/miniml/test_exn test_exp
```

This builds the extended MiniML and executes the test cases in `src/miniml/test_exn`. Expected output is in the `test_exp` file.

To run MiniML after building:

```
./miniml.exe
```

4 Git Commits

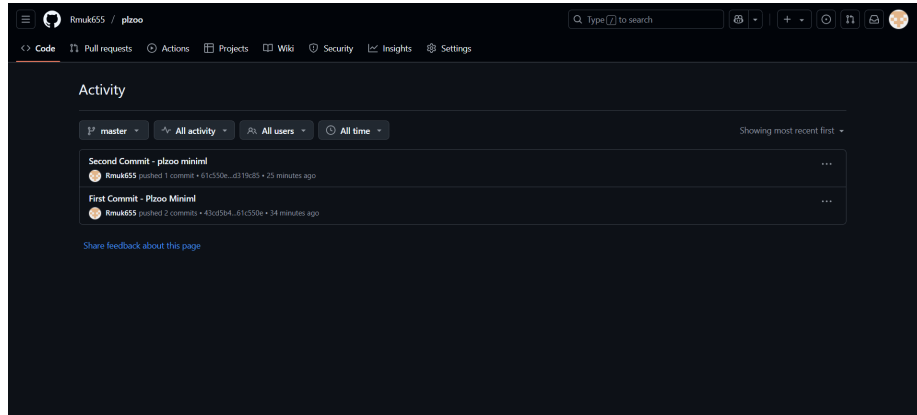


Figure 1: Git Commits

5 Limitations of My Library

The current implementation of MiniML has a few unsupported cases, particularly when it comes to exception handling and certain language constructs. The following are the limitations of My Library when used for calculations:

- **Incorrect argument type for `GenericException`:** The argument provided to the `GenericException` must be an integer. If a string is used instead, the compiler gives an error. This happens because meaningful error messages for such cases have not been hardcoded.
- **Unsupported Wildcard (`_`) in Expressions:** Expressions like

```
try 1 + 1 with _ -> 0;;
```

are not supported in MiniML (unlike OCaml) and result in a syntax error.

- **Unsupported Multiple Handlers for Exceptions:** Expressions with multiple exception handlers such as

```
try raise GenericException 100 with  
  DivisionByZero -> 0  
| GenericException n -> n ;
```

are not supported in MiniML.

- **Invalid Exception Handler Type:** Expressions like `try raise DivisionByZero with DivisionByZero -> "error" ;;` give an error. The `DivisionByZero` exception handler is incorrectly returning a string, while it should return an integer. The type checker should catch this and provide a helpful error message.

6 Verification Approach

My verification approach focuses on handling various test cases that verify if each implemented functionality is correct. The approach involves running test cases that ensure the correct behavior of language features in various contexts. These test cases cover both basic and complex scenarios, including edge cases for exceptions, nested functions, and multiple `try ... with` blocks.

Test Case Breakdown

- **Simple Arithmetic and Function Tests:**

```
3 + (if 5 < 6 then 10 else 100) ;;
let x = 14 ;;
let fact = fun f (n : int) : int is if n = 0 then 1
    ↪ else n * f (n-1) ;;
fact 10 ;;
```

- **Basic Exception Raising:**

```
raise DivisionByZero ;;
raise GenericException(1) ;;
raise GenericException(-2) ;;
raise GenericException 3 ;;
raise GenericException -4 ;;
```

- **Basic Exception Handling:**

```
try raise DivisionByZero with DivisionByZero -> 5 ;;
try raise DivisionByZero with DivisionByZero -> -5 ;;
try raise (DivisionByZero) with DivisionByZero -> 6 ;;
try raise (DivisionByZero) with (DivisionByZero) -> -6
    ↪ ;;
try -7 / 0 with (DivisionByZero) -> -7 ;;
try 7 / 0 with DivisionByZero -> 7 ;;
try 8 + 0 with GenericException 108 -> 118 ;;
try 9 / 0 with GenericException(42) -> 9 ;;
try raise GenericException(101) with
    ↪ GenericException(101) -> 10 ;;
try raise GenericException(-101) with
    ↪ GenericException(-101) -> -10 ;;
```

```
try raise GenericException(102) with
  ↪ GenericException(201) -> 11 ;;
```

- **Nested try ... with Blocks:**

```
try try raise DivisionByZero with DivisionByZero -> 17
  ↪ with DivisionByZero -> 108 ;;
try (try raise DivisionByZero with DivisionByZero ->
  ↪ 109/0) with DivisionByZero -> 18 ;;
try (try 10/0 with DivisionByZero -> 109/0) with
  ↪ DivisionByZero -> -18 ;;
try (try raise DivisionByZero with
  ↪ GenericException(201) -> 201) with DivisionByZero
  ↪ -> 19 ;;
try (try raise DivisionByZero with
  ↪ GenericException(202) -> 202) with
  ↪ GenericException(202) -> 20 ;;
```

- **More Complex Nested try ... with Blocks:**

```
try try raise (DivisionByZero) with (DivisionByZero) ->
  ↪ raise (GenericException 203)
  with (GenericException 203) -> 21 ;;

try try 10/0 with (DivisionByZero) -> raise
  ↪ (GenericException 203)
  with (GenericException 203) -> -21 ;;

try try raise DivisionByZero with DivisionByZero ->
  ↪ raise (GenericException 1)
  with GenericException 1 -> -21 ;;
```

- **Function with Exception Handling:**

```
let fact = fun f (n : int) : int is
  if n < 0 then raise GenericException(-99)
  else if n = 0 then 1
  else n * f (n - 1) ;;

try fact 5 with GenericException(1) -> 30 ;;
try fact (-3) with GenericException(-99) -> -30 ;;
```

- **More Complex Function with Exception Handling:**

```
let safediv = fun f (n : int) : int is
  try 62 / n with DivisionByZero -> -31 ;;

safediv 2 ;;
safediv 0 ;;
```

```

let fib = fun f (n : int) : int is
  if n < 21 then
    if n < 2 then n
    else f(n - 1) + f(n - 2)
  else raise GenericException(-32) ;;

try fib 10 with GenericException(-32) -> 33 ;;
try fib 30 with GenericException(-32) -> -33 ;;

let tryadd = fun f(x : int) : int is
  try if x < 0 then raise GenericException(-100) else
    ↪ x + 29
  with GenericException(-100) -> -34 ;;

tryadd 5 ;;
tryadd (-2) ;;

```

7 Key Learnings

Some of my key learnings from the project are

- Integrating OCaml and Dune: I learnt to use programming languages like OCaml, Dune. Dune helped automate the build process, managing multiple modules and dependencies. Understanding how Dune integrates with OCaml helped simplify compiling and running the project.
- Concepts of Compiler: I understood how the compiler of any programming language works including its key components like lexer, parser, compiler, machine code, type checker, etc. The lexer tokenizes the commands and sends it to the parser which checks if they follow the rules of the language, sending them to the compiler which compiles it to a set of machine instructions and sends it to the machine which performs the final calculations.
- Learning to handle Keywords and Tokens: Managing keywords like try, with, raise, and exceptions (DivisionByZero, GenericException) was crucial. This required careful design of the lexer rules to properly recognize these keywords and associate them with the right tokens for further processing.
- Implementing Exception Handling: The project focused on handling exceptions, including the definitions of DivisionByZero and GenericException, which were incorporated as part of the language syntax. The exception-handling mechanism, especially with try ... with blocks, was a key challenge.

- Order of Tokens: The order of patterns in the lexer matters a lot. Tokens like `DIVIDE(/)`, `RAISE`, and `TRY` had to be handled before general expressions, as specific tokens must be recognized first to prevent ambiguity in parsing.
- The role of keywords in the lexer: Ensuring that keywords such as `try`, `with`, `raise`, `DivisionByZero`, and `GenericException` were identified correctly was a core issue. Mistakes in this area resulted in incorrect tokenization and parsing.
- Testing and Debugging: Each test case revealed new bugs, especially around exception handling. For example, handling exceptions such as `DivisionByZero` or `GenericException` in various forms (parenthesized vs non-parenthesized) uncovered subtle issues in token recognition, parsing, and exception matching. This reinforced the need for comprehensive test coverage.